Feature-Flag Driven UI

Problem: Build a UI whose features turn on and off from remote config: a flag store, a `useFlag` hook / `<Feature>` component, user targeting, percentage rollout, and an admin panel to manage flags.

- **Tech stack:** React (Vite, JavaScript), default since none was given
- **Time limit:** 60 minutes, default since none was given
- **Often asked at:** Atlassian, Swiggy

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements**: functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases**: every user action and what must happen. This becomes your test checklist.
3. **Design the architecture**: components / modules and their single responsibility.
4. **Design the state**: the minimum state, who owns it, what is derived.
5. **Design data structures and contracts**: example data, API shapes, mocks.
6. **Trace the data flow** for one key interaction end to end.
7. **Find the hard parts** and decide how to solve them before coding.
8. **Build in layers**: simplest working version first, then one working layer at a time.
9. **Cover states and edge cases**: loading, empty, error, success and problem-specific ones.
10. **Wrap up**: trade-offs and what you would add with more time.

Time budget (60 minutes)

Phase	Minutes
Requirements, questions, assumptions	5
Use cases, architecture, state, contracts	8
Core: evaluation function + provider + `useFlag` / `<Feature>`	15
Gated demo UI + admin panel (toggle, rollout, overrides)	17
Loading / error / cache / optimistic rollback	10
Edge cases, polish, walkthrough	5

Tip: think out loud

Say every decision and why: "Flags default to OFF while loading, because the safe fallback is the old experience." The interviewer grades your reasoning as much as the code. If you are stuck, say what you are trying, and what you would do with more time.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Load flag configuration from an API (mocked) when the app starts.
- Any component can ask "is flag X on for the current user?" via `useFlag('x')` or wrap UI in `<Feature flag="x" fallback={...}>`.
- A flag can be: globally on/off (kill switch), targeted to roles, rolled out to a percentage of users, and depend on another flag.
- The same user always gets the same answer for a percentage rollout (no flip-flopping on refresh).
- Admin panel: list and search flags, toggle on/off, change rollout %, see the result and the *reason* for the current user.
- Local overrides (force on / force off) for developers and QA, kept across reloads.
- Switch the current user to preview what different users see.

Non-functional requirements

Area	What it means here
Performance	Evaluate all flags once per change (memoized), not on every `useFlag` call; slider sends one request per drag, not per pixel.
Scalability	Evaluation is a pure function, so it can move to the server or a worker later; flags are data, so new flags need no UI code.
Error handling	Failed fetch never breaks the app: fall back to cached flags, else to safe defaults (OFF). Failed update rolls back.
Accessibility / usability	Real `<input type="checkbox" role="switch">`, `<select>`, labelled range inputs, `role="alert"` for errors, `aria-busy` skeleton.
Reusability	`FlagProvider`, `useFlag`, `<Feature>` are app-agnostic; any screen can use them.
Extensibility	New rule types (country, app version, A/B variants) are one more step inside `evaluateFlag`.

Clarifying questions to ask

1. Is this a client SDK (consume flags) or also the admin dashboard (manage flags)? Both in one screen?
2. Where does evaluation happen: on the client with the full config, or does the server return already-evaluated booleans per user?
3. What targeting rules matter: roles, percentage, user ids, country? Are there multivariate flags (A/B variants) or only booleans?
4. What should users see while flags are loading or if the flag service is down?
5. Do flag changes need to reach open sessions live (polling / SSE), or on next load?
6. Do developers need local overrides, and should overrides be possible in production?

Assumptions we will build with

- Boolean flags only; rules: `enabled` kill switch, `roles` targeting, `rollout` 0-100%, optional `requires` (prerequisite flag).
- Evaluation happens on the client from the full config (simple, and the admin panel needs the config anyway).
- Mock API with 400-800 ms latency and a "fail next request" switch to demo errors; the mock "server" persists in `localStorage`.
- Unknown / loading / failed flags are **OFF**. Last-known flags are cached in `localStorage` and used on the next boot.
- Changes reach the UI on reload / Refresh; live push is a "with more time" item.
- Current user comes from a user switcher (stands in for auth).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
App first load, no cache	Skeleton in the store preview, admin shows "Refreshing…"; after fetch, gated UI renders.
App load with cached flags	Store renders instantly from cache (no flicker), fetch refreshes it in the background.
Flags fetch fails, cache exists	Notice "using the last known values" + Retry; UI keeps working.
Flags fetch fails, no cache	Notice "showing the default experience" + Retry; all flags OFF.
`useFlag('unknown_key')`	Returns `false`, never throws.
Switch user	All flags re-evaluated for that user; store preview and "For user" column update.
User with `beta` role	Sees beta banner; `user` role does not (reason: role not targeted).
Rollout 50%, user bucket 58	Flag OFF with reason `bucket 58 ≥ 50%`; same answer every reload.
Toggle flag in admin	UI flips instantly (optimistic), request saves it; on failure it flips back and shows an error.
Drag rollout slider	Number updates live; one PATCH on release.
Turn off a parent flag	Child flag (`requires`) turns OFF with reason `needs "parent"`.
Set override Force on / off	Wins over every server rule, persists across reload; "Clear overrides" removes all.
Search flags	Filters by key or description; "No flags match" when empty.
Circular prerequisites (A needs B, B needs A)	Both OFF, no infinite loop.

---

3. Design the architecture

Module	Responsibility
`flags/evaluate.js`	Pure functions: hash a user into a bucket, evaluate one flag, evaluate all flags with prerequisites.
`flags/flagApi.js`	Mock backend: `fetchFlags()`, `updateFlag(key, patch)` with latency and failures.
`flags/storage.js`	Safe `localStorage` JSON read/write.
`flags/FlagContext.js`	Context + `useFlags()` + `useFlag(key)` hooks.
`flags/FlagProvider.jsx`	Owns flags, status, error, overrides; fetch, cache, optimistic update; memoized results.
`flags/Feature.jsx`	Declarative gate: render children if on, fallback if off.
`components/Storefront.jsx`	Demo UI gated by flags (banner, nav link, checkout, upsell, dark mode).
`components/AdminPanel.jsx`	Flag table: search, toggle, rollout slider, targeting, result + reason, override.
`App.jsx`	User switcher; wraps everything in `FlagProvider`.

```
App (current user)
└── FlagProvider (flags, status, overrides → results)
    ├── Storefront
    │   ├── <Feature flag="beta_banner">
    │   ├── <Feature flag="admin_reports">
    │   └── <Feature flag="new_checkout" fallback={classic}>
    │       └── <Feature flag="checkout_upsell">
    └── AdminPanel
        └── row per flag → RolloutSlider

FlagProvider ──uses──> flagApi (network)   evaluate (pure)   storage (cache)
```

Start simple: a provider with a `flags` array and `useFlag` that reads `flag.enabled`. Only then add targeting, rollout, prerequisites, admin, overrides and caching, each as its own layer.

---

4. Design the state / data model

State or field	Purpose
`userId` (App)	Which user we evaluate for.
`flags` (Provider)	The flag config from the server (seeded from cache).
`status` (Provider)	`loading` / `ready` / `error` for the latest fetch.
`error` (Provider)	Message for a failed fetch or update.

State or field	Purpose
`overrides` (Provider)	`{ [key]: true / false }` local forced values, persisted.
`requestId` (ref)	Ignore responses from an older fetch.
`draft` (RolloutSlider)	Slider value while dragging, `null` when idle.
`query` (AdminPanel)	Search text.

```
const [flags, setFlags] = useState(() => readJSON('ff:cache', []));
const [status, setStatus] = useState('loading'); // loading | ready | error
const [error, setError] = useState(null);
const [overrides, setOverrides] = useState(() => readJSON('ff:overrides', {}));
const requestId = useRef(0);

const results = useMemo(() => evaluateAll(flags, user, overrides), [flags, user, overrides]);
```

Senior note:

- **Derived, not stored:** `results` (on/off + reason per flag), the visible (searched) flag list, the user object from `userId`, the "Clear overrides" disabled state. Storing `results` would go stale the moment the user, a flag or an override changes.
- **Ownership:** the provider owns everything shared (flags, overrides, status). UI-only state stays local: the slider draft lives in the slider, the search query in the admin panel. The current user is owned above the provider, because auth decides it, not the flag system.

---

5. Design the data structures and contracts

Example data

```
const flags = [
  { key: 'new_checkout',    description: 'Redesigned one-page checkout', enabled: true, rollout: 50,
roles: [],        requires: null },
  { key: 'checkout_upsell', description: 'Gift-wrap upsell',             enabled: true, rollout: 100,
roles: [],        requires: 'new_checkout' },
  { key: 'beta_banner',     description: 'Banner for beta testers',      enabled: true, rollout: 100,
roles: ['beta'],  requires: null },
  { key: 'admin_reports',   description: 'Reports tab in the nav',       enabled: true, rollout: 100,
roles: ['admin'], requires: null },
  { key: 'dark_mode',       description: 'Dark theme for the store',     enabled: false, rollout: 100,
roles: [],       requires: null },
];
const user = { id: 'u-101', name: 'Asha', role: 'user' };
```

Why: `key` is the **stable id**: code references it (`useFlag('new_checkout')`), the API path uses it, React uses it as list key. It never changes; the description can. `requires` refers to another key, so prerequisites are data, not code. Users have a stable `id` because the rollout bucket is computed from it.

API contracts

Call	Input	Output
`GET /flags` → `fetchFlags()`	none	`Flag[]`
`PATCH /flags/:key` → `updateFlag(key, patch)`	`{ enabled?: boolean, rollout?: number }`	updated `Flag`
Errors		rejects with `Error(message)`; 404-like for unknown key

```
PATCH /flags/new_checkout
{ "rollout": 70 }

200 OK
{ "key": "new_checkout", "description": "Redesigned one-page checkout",
  "enabled": true, "rollout": 70, "roles": [], "requires": null }
```

Hook / component contracts

API	Contract
`useFlag(key): boolean`	`true` only if the flag exists, was evaluated ON for the current user. Never throws for unknown keys.
`<Feature flag fallback children>`	Renders `children` when ON, `fallback` (default `null`) when OFF.
`useFlags()`	`{ user, flags, results, status, error, overrides, reload, updateFlag, setOverride, clearOverrides, dismissError }`; throws if used outside the provider.
`evaluateFlag(flag, user, override) → { on, reason }`	Pure; rule order: override → kill switch → roles → rollout.

---

6. Trace the data flow

Admin toggles `new_checkout` off while viewing as Meera:

```
Click "Enabled" checkbox on new_checkout row
      |
      ▼
```

```
updateFlag('new_checkout', { enabled: false })
      |
      ▼
setFlags: optimistic patch, remember previous { enabled: true }
      |
      ▼
useMemo → evaluateAll(flags, Meera, overrides)
   new_checkout   → OFF "flag is off"
   checkout_upsell → its own rules pass, but requires new_checkout → OFF "needs new_checkout"
      |
      ▼
Context value changes → Storefront re-renders
   <Feature flag="new_checkout"> shows fallback "Checkout (3 steps)"; upsell disappears
      |
      ▼
api.updateFlag(...) (400-800 ms)
   ├── success → replace flag with server copy, write cache
   └── failure → restore { enabled: true }, set error → alert "Could not update…" + Dismiss
```

---

7. Important technical challenges

Challenge 1: Sticky percentage rollout

Problem: "Show the new checkout to 50% of users." If you use `Math.random() < 0.5`, Asha sees the new checkout, reloads, and gets the old one. Analytics and support break.

Solution: Hash `flagKey:userId` to a bucket 0-99 (FNV-1a). Same input, same bucket, every time and on every device. Including the flag key means a user isn't in the "first 10%" of *every* experiment. Raising the rollout from 50% to 70% only *adds* users (buckets 50-69); nobody who had the feature loses it.

```
export function bucketOf(userId, flagKey) {
  let h = 2166136261;
  for (const ch of `${flagKey}:${userId}`) {
    h ^= ch.charCodeAt(0);
    h = Math.imul(h, 16777619);
  }
  return (h >>> 0) % 100;
}
// on when bucket < rollout
```

Challenge 2: Rule order, prerequisites and cycles

Problem: `checkout_upsell` only makes sense inside `new_checkout`. If the parent is off for Asha but the child is evaluated alone, the app could try to show an upsell in a checkout that doesn't exist. And a bad config (A

needs B, B needs A) must not freeze the tab.

Solution: Evaluate all flags in one pass with memoization; walk `requires` with a `visiting` set to detect cycles. Return a `reason` with every result, so the admin panel (and support) can explain *why* a user sees what they see.

```
const visit = (key) => {
  if (results[key]) return results[key];
  if (visiting.has(key)) return { on: false, reason: 'circular prerequisite' };
  visiting.add(key);
  let res = evaluateFlag(byKey[key], user, overrides[key]);
  if (res.on && overrides[key] === undefined && byKey[key].requires && !visit(byKey[key].requires).on)
{
    res = { on: false, reason: `needs "${byKey[key].requires}"` };
  }
  visiting.delete(key);
  return (results[key] = res);
};
```

Challenge 3: Loading, failure and flicker

Problem: On first render no flags are loaded. If everything defaults to OFF, a user in the new checkout sees the classic button for 600 ms and then it swaps (layout shift, maybe a mis-click). If the flag service is down, the app must still work.

Solution:

- Default OFF = the safe, old experience.
- Cache the last good config in `localStorage` and start state from it (stale-while-revalidate): returning users get the right UI on the first frame.
- First-ever visit (no cache): show a skeleton instead of the wrong UI.
- Fetch failure: keep cached flags (or defaults) and show a Retry notice. Ignore out-of-order responses with a request id.

```
const [flags, setFlags] = useState(() => readJSON(CACHE_KEY, []));
...
const id = ++requestId.current;
const data = await api.fetchFlags();
if (id !== requestId.current) return; // newer reload started
setFlags(data); writeJSON(CACHE_KEY, data); setStatus('ready');

// Storefront
if (!flags.length && status === 'loading') return <Skeleton />;
```

Challenge 4: Optimistic admin updates without spamming the API

Problem: Toggles should feel instant, but the network can fail. A rollout slider fires `onChange` for every step, which would be 20 PATCH requests per drag.

Solution: Patch local state first, remember only the fields we changed, and restore just those on failure. Keep the slider value in a local `draft` while dragging and commit once on pointer up / key up / blur.

```
setFlags((fs) => fs.map((f) => {
  if (f.key !== key) return f;
  previous = Object.fromEntries(Object.keys(patch).map((k) => [k, f[k]]));
  return { ...f, ...patch };
}));
try { const saved = await api.updateFlag(key, patch); /* replace + cache */ }
catch (e) { setFlags((fs) => fs.map((f) => (f.key === key ? { ...f, ...previous } : f)));
setError(...); }
```

---

8. Implement in the correct order

#	Step	Working result
1	Hard-code a flags array, `FlagProvider` + `useFlag` reading `flag.enabled`	One gated button toggles in code
2	`<Feature flag fallback>` component	Declarative gating in JSX
3	Mock API `fetchFlags` with latency; `status` loading/ready/error	Flags come from "server"
4	`evaluateFlag`: kill switch + roles + user switcher	Different users see different UI
5	`bucketOf` + rollout %	Sticky percentage rollout
6	`evaluateAll` with `requires` + cycle guard, `reason` strings	Prerequisites work, explainable results
7	Admin table: list, search, result + reason column	Visibility into every flag
8	`updateFlag` optimistic toggle + rollback + error alert	Admin can change flags safely
9	Rollout slider with draft + commit on release	One request per drag
10	Local overrides (persisted) + Clear overrides	QA can force states
11	Cache last flags, skeleton on first load, Retry on error, stale-response guard	No flicker, resilient
12	Accessibility labels, responsive layout, node self-check of `evaluate.js`	Polished and tested

---

9. Cover all states and edge cases

State / edge case	Handling
Initial, no cache	Skeleton (`aria-busy`), admin "Refreshing…"
Initial, with cache	Render cached flags immediately, refresh in background
Loading (refresh)	Refresh button disabled with "Refreshing…"
Success	Gated UI + table rendered
Empty (no flags configured)	"No flags configured."
Empty search result	`No flags match "query".`
Fetch error, cache exists	Notice: using last known values + Retry
Fetch error, no cache	Notice: default experience (all OFF) + error + Retry
Update error	Rollback the changed fields + alert with Dismiss
Unknown flag key in code	`useFlag` returns `false`
Prerequisite missing / off	Child OFF, reason `needs "x"`
Circular prerequisites	Both OFF, reason "circular prerequisite"
Rollout 0% / 100%	0% nobody (bucket ≥ 0), 100% everyone targeted
Override vs. server	Override always wins, skips prerequisite check, persisted
Out-of-order fetch responses (StrictMode double effect, double Refresh)	Request id drops stale responses
`localStorage` blocked / corrupt JSON	`readJSON` / `writeJSON` swallow errors, fall back to defaults
Hook used outside provider	Clear error thrown by `useFlags`
Switching user	Results re-derive instantly; no refetch needed

10. Final structure

```
src/
├── main.jsx
├── App.jsx                    user switcher + FlagProvider
├── index.css
├── flags/
│   ├── evaluate.js            bucketOf, evaluateFlag, evaluateAll (pure)
│   ├── evaluate.test.mjs      node assert self-check
│   ├── flagApi.js             mock GET / PATCH with latency + failures
```

```
│   ├── storage.js             safe localStorage JSON
│   ├── FlagContext.js         context, useFlags, useFlag
│   ├── FlagProvider.jsx       state, fetch, cache, optimistic update
│   └── Feature.jsx            <Feature flag fallback>
└── components/
    ├── Storefront.jsx         demo UI gated by flags
    └── AdminPanel.jsx         flag table + RolloutSlider
```

---

11. Full working code

src/flags/evaluate.js

```
// FNV-1a hash: the same user + flag always lands in the same bucket (0-99),
// so a 30% rollout shows the feature to the same 30% of users on every visit.
export function bucketOf(userId, flagKey) {
  let h = 2166136261;
  for (const ch of `${flagKey}:${userId}`) {
    h ^= ch.charCodeAt(0);
    h = Math.imul(h, 16777619);
  }
  return (h >>> 0) % 100;
}

// One flag, ignoring prerequisites. Order matters: override > kill switch > targeting > rollout.
export function evaluateFlag(flag, user, override) {
  if (!flag) return { on: false, reason: 'unknown flag' };
  if (override !== undefined) return { on: override, reason: 'local override' };
  if (!flag.enabled) return { on: false, reason: 'flag is off' };
  if (flag.roles.length && !flag.roles.includes(user.role)) {
    return { on: false, reason: `role "${user.role}" not targeted` };
  }
  const bucket = bucketOf(user.id, flag.key);
  if (bucket >= flag.rollout) return { on: false, reason: `bucket ${bucket} ≥ ${flag.rollout}%` };
  return { on: true, reason: flag.rollout < 100 ? `bucket ${bucket} < ${flag.rollout}%` : 'on for
everyone targeted' };
}

// All flags at once, so prerequisites ("requires") resolve once and cycles cannot loop forever.
export function evaluateAll(flags, user, overrides = {}) {
  const byKey = Object.fromEntries(flags.map((f) => [f.key, f]));
  const results = {};
  const visiting = new Set();

  const visit = (key) => {
```

```
    if (results[key]) return results[key];
    if (visiting.has(key)) return { on: false, reason: 'circular prerequisite' };
    visiting.add(key);
    const flag = byKey[key];
    let res = evaluateFlag(flag, user, overrides[key]);
    if (res.on && overrides[key] === undefined && flag.requires && !visit(flag.requires).on) {
      res = { on: false, reason: `needs "${flag.requires}"` };
    }
    visiting.delete(key);
    return (results[key] = res);
  };

  flags.forEach((f) => visit(f.key));
  return results;
}
```

src/flags/storage.js

```
// localStorage can throw (private mode, quota) or hold bad JSON: never let that crash the app.
export function readJSON(key, fallback) {
  try {
    return JSON.parse(localStorage.getItem(key)) ?? fallback;
  } catch {
    return fallback;
  }
}

export function writeJSON(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
  } catch {
    // storage full or blocked: the app still works, it just won't remember
  }
}
```

src/flags/flagApi.js

```
import { readJSON, writeJSON } from './storage.js';

// Mock backend: the "server" keeps its data in localStorage so admin edits survive a reload.
const SERVER_KEY = 'ff:server';

const SEED = [
  { key: 'new_checkout', description: 'Redesigned one-page checkout', enabled: true, rollout: 50,
roles: [], requires: null },
```

```
  { key: 'checkout_upsell', description: 'Gift-wrap upsell inside the new checkout', enabled: true, rollout: 100, roles: [], requires: 'new_checkout' },
  { key: 'beta_banner', description: 'Banner thanking beta testers', enabled: true, rollout: 100, roles: ['beta'], requires: null },
  { key: 'admin_reports', description: 'Reports tab in the nav', enabled: true, rollout: 100, roles: ['admin'], requires: null },
  { key: 'dark_mode', description: 'Dark theme for the store', enabled: false, rollout: 100, roles: [], requires: null },
];

let db = readJSON(SERVER_KEY, SEED);
let failNext = false;

export const failNextRequest = () => {
  failNext = true;
};

const delay = (ms) => new Promise((r) => setTimeout(r, ms));

async function request(handler) {
  await delay(400 + Math.random() * 400);
  if (failNext) {
    failNext = false;
    throw new Error('Network error (simulated)');
  }
  return structuredClone(handler());
}

// GET /flags -> Flag[]
export const fetchFlags = () => request(() => db);

// PATCH /flags/:key { enabled?, rollout? } -> Flag
export const updateFlag = (key, patch) =>
  request(() => {
    const flag = db.find((f) => f.key === key);
    if (!flag) throw new Error(`Unknown flag "${key}"`);
    db = db.map((f) => (f.key === key ? { ...f, ...patch } : f));
    writeJSON(SERVER_KEY, db);
    return db.find((f) => f.key === key);
  });
```

src/flags/FlagContext.js

```
import { createContext, useContext } from 'react';

export const FlagContext = createContext(null);
```

```
export function useFlags() {
  const ctx = useContext(FlagContext);
  if (!ctx) throw new Error('useFlags must be used inside <FlagProvider>');
  return ctx;
}

// Unknown, loading or failed flags are all OFF: the safe default is the old experience.
export function useFlag(key) {
  return useFlags().results[key]?.on ?? false;
}
```

src/flags/FlagProvider.jsx

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import * as api from './flagApi.js';
import { evaluateAll } from './evaluate.js';
import { FlagContext } from './FlagContext.js';
import { readJSON, writeJSON } from './storage.js';

const CACHE_KEY = 'ff:cache';
const OVERRIDES_KEY = 'ff:overrides';

export default function FlagProvider({ user, children }) {
  // Start from the last known flags so a returning user sees no flicker (stale-while-revalidate).
  const [flags, setFlags] = useState(() => readJSON(CACHE_KEY, []));
  const [status, setStatus] = useState('loading'); // loading | ready | error
  const [error, setError] = useState(null);
  const [overrides, setOverrides] = useState(() => readJSON(OVERRIDES_KEY, {}));
  const requestId = useRef(0);

  const reload = useCallback(async () => {
    const id = ++requestId.current;
    setStatus('loading');
    setError(null);
    try {
      const data = await api.fetchFlags();
      if (id !== requestId.current) return; // a newer reload started: drop this stale response
      setFlags(data);
      writeJSON(CACHE_KEY, data);
      setStatus('ready');
    } catch (e) {
      if (id !== requestId.current) return;
      setError(e.message);
      setStatus('error');
    }
```

```
  }, []);

  useEffect(() => {
    reload();
  }, [reload]);

  // Optimistic: update the UI now, roll back only the fields we touched if the server says no.
  const updateFlag = useCallback(async (key, patch) => {
    let previous;
    setFlags((fs) =>
      fs.map((f) => {
        if (f.key !== key) return f;
        previous = Object.fromEntries(Object.keys(patch).map((k) => [k, f[k]]));
        return { ...f, ...patch };
      }),
    );
    try {
      const saved = await api.updateFlag(key, patch);
      setFlags((fs) => {
        const next = fs.map((f) => (f.key === key ? saved : f));
        writeJSON(CACHE_KEY, next);
        return next;
      });
    } catch (e) {
      setFlags((fs) => fs.map((f) => (f.key === key ? { ...f, ...previous } : f)));
      setError(`Could not update "${key}": ${e.message}`);
    }
  }, []);

  const setOverride = useCallback((key, value) => {
    setOverrides((o) => {
      const next = { ...o };
      if (value === undefined) delete next[key];
      else next[key] = value;
      writeJSON(OVERRIDES_KEY, next);
      return next;
    });
  }, []);

  const clearOverrides = useCallback(() => {
    setOverrides({});
    writeJSON(OVERRIDES_KEY, {});
  }, []);

  const results = useMemo(() => evaluateAll(flags, user, overrides), [flags, user, overrides]);

  const value = useMemo(
```

```
    () => ({ user, flags, results, status, error, overrides, reload, updateFlag, setOverride,
clearOverrides, dismissError: () => setError(null) }),
    [user, flags, results, status, error, overrides, reload, updateFlag, setOverride, clearOverrides],
  );

  return <FlagContext.Provider value={value}>{children}</FlagContext.Provider>;
}
```

src/flags/Feature.jsx

```
import { useFlag } from './FlagContext.js';

// <Feature flag="new_checkout" fallback={<OldCheckout />}><NewCheckout /></Feature>
export default function Feature({ flag, fallback = null, children }) {
  return useFlag(flag) ? children : fallback;
}
```

src/components/Storefront.jsx

```
import Feature from '../flags/Feature.jsx';
import { useFlag, useFlags } from '../flags/FlagContext.js';

export default function Storefront() {
  const { user, flags, status, error, reload } = useFlags();
  const dark = useFlag('dark_mode');

  // First visit, nothing cached yet: show a skeleton instead of flashing the old UI then swapping.
  if (!flags.length && status === 'loading') {
    return (
      <section className="card store" aria-busy="true">
        <div className="skeleton" />
        <div className="skeleton short" />
        <div className="skeleton" />
      </section>
    );
  }

  return (
    <section className={`card store${dark ? ' dark' : ''}`}>
      <h2>
        Store preview <small>as {user.name}</small>
      </h2>

      {status === 'error' && (
        <p className="notice" role="alert">
```

```
          {flags.length ? 'Flags could not refresh, using the last known values.' : 'Flags could not
load, showing the default experience.'}{' '}
          <button onClick={reload}>Retry</button>
          {!flags.length && <span className="muted"> ({error})</span>}
        </p>
      )}

      <Feature flag="beta_banner">
        <div className="banner">Thanks for testing the beta, {user.name}!</div>
      </Feature>

      <nav className="store-nav">
        <a href="#home">Home</a>
        <a href="#orders">Orders</a>
        <Feature flag="admin_reports">
          <a href="#reports">Reports</a>
        </Feature>
      </nav>

      <div className="product">
        <div className="thumb" aria-hidden="true" />
        <div>
          <h3>Wireless headphones</h3>
          <p className="muted">₹2,499</p>
          <Feature flag="new_checkout" fallback={<button className="btn">Add to cart → Checkout (3
steps)</button>}>
            <button className="btn primary">Buy now, one-page checkout</button>
            <Feature flag="checkout_upsell">
              <label className="upsell">
                <input type="checkbox" /> Add gift wrap for ₹49
              </label>
            </Feature>
          </Feature>
        </div>
      </div>
    </section>
  );
}
```

src/components/AdminPanel.jsx

```
import { useState } from 'react';
import { useFlags } from '../flags/FlagContext.js';
import { failNextRequest } from '../flags/flagApi.js';

// Local draft while dragging, one request on release (not one per pixel).
```

```
function RolloutSlider({ flag, onCommit }) {
  const [draft, setDraft] = useState(null);
  const commit = () => {
    if (draft !== null && draft !== flag.rollout) onCommit(draft);
    setDraft(null);
  };
  return (
    <span className="rollout">
      <input
        type="range"
        min="0"
        max="100"
        step="5"
        value={draft ?? flag.rollout}
        aria-label={`${flag.key} rollout percentage`}
        onChange={(e) => setDraft(Number(e.target.value))}
        onPointerUp={commit}
        onKeyUp={commit}
        onBlur={commit}
      />
      <output>{draft ?? flag.rollout}%</output>
    </span>
  );
}

const OVERRIDE_OPTIONS = { default: undefined, on: true, off: false };

export default function AdminPanel() {
  const { flags, results, overrides, status, error, reload, updateFlag, setOverride, clearOverrides,
dismissError, user } = useFlags();
  const [query, setQuery] = useState('');

  const q = query.trim().toLowerCase();
  const visible = flags.filter((f) => f.key.includes(q) || f.description.toLowerCase().includes(q));

  return (
    <section className="card admin">
      <header className="admin-head">
        <h2>Feature flags</h2>
        <input type="search" placeholder="Search flags" aria-label="Search flags" value={query}
onChange={(e) => setQuery(e.target.value)} />
        <button onClick={reload} disabled={status === 'loading'}>
          {status === 'loading' ? 'Refreshing…' : 'Refresh'}
        </button>
        <button onClick={clearOverrides} disabled={!Object.keys(overrides).length}>
          Clear overrides
        </button>
```

```
        <button onClick={failNextRequest} title="The next API call will fail">
          Fail next request
        </button>
      </header>

      {error && status !== 'error' && (
        <p className="notice" role="alert">
          {error} <button onClick={dismissError}>Dismiss</button>
        </p>
      )}

      {!flags.length && status !== 'loading' ? (
        <p className="muted">No flags configured.</p>
      ) : !visible.length ? (
        <p className="muted">No flags match "{query}".</p>
      ) : (
        <table>
          <thead>
            <tr>
              <th>Flag</th>
              <th>Enabled</th>
              <th>Rollout</th>
              <th>Targeting</th>
              <th>For {user.name}</th>
              <th>Override</th>
            </tr>
          </thead>
          <tbody>
            {visible.map((flag) => {
              const res = results[flag.key];
              const override = overrides[flag.key];
              return (
                <tr key={flag.key}>
                  <td>
                    <code>{flag.key}</code>
                    <div className="muted">{flag.description}</div>
                  </td>
                  <td>
                    <label className="switch">
                      <input
                        type="checkbox"
                        role="switch"
                        checked={flag.enabled}
                        aria-label={`${flag.key} enabled`}
                        onChange={(e) => updateFlag(flag.key, { enabled: e.target.checked })}
                      />
                      <span>{flag.enabled ? 'On' : 'Off'}</span>
```

```
                  </label>
                </td>
                <td>
                  <RolloutSlider flag={flag} onCommit={(rollout) => updateFlag(flag.key, { rollout
})} />
                </td>
                <td className="muted">
                  {flag.roles.length ? `roles: ${flag.roles.join(', ')}` : 'everyone'}
                  {flag.requires && <div>needs {flag.requires}</div>}
                </td>
                <td>
                  <span className={`pill ${res?.on ? 'on' : 'off'}`}>{res?.on ? 'ON' : 'OFF'}</span>
                  <div className="muted">{res?.reason}</div>
                </td>
                <td>
                  <select
                    aria-label={`${flag.key} override`}
                    value={override === undefined ? 'default' : override ? 'on' : 'off'}
                    onChange={(e) => setOverride(flag.key, OVERRIDE_OPTIONS[e.target.value])}
                  >
                    <option value="default">Server</option>
                    <option value="on">Force on</option>
                    <option value="off">Force off</option>
                  </select>
                </td>
              </tr>
            );
          })}
        </tbody>
      </table>
    )}
  </section>
 );
}
```

src/App.jsx

```
import { useState } from 'react';
import FlagProvider from './flags/FlagProvider.jsx';
import Storefront from './components/Storefront.jsx';
import AdminPanel from './components/AdminPanel.jsx';

// In a real app the user comes from auth; here we switch users to see targeting and rollout work.
const USERS = [
  { id: 'u-101', name: 'Asha', role: 'user' },
  { id: 'u-202', name: 'Ravi', role: 'beta' },
```

```
  { id: 'u-303', name: 'Meera', role: 'admin' },
  { id: 'u-404', name: 'Kabir', role: 'user' },
];

export default function App() {
  const [userId, setUserId] = useState(USERS[0].id);
  const user = USERS.find((u) => u.id === userId);

  return (
    <main className="app">
      <header className="top">
        <h1>Feature-flag driven UI</h1>
        <label>
          Viewing as{' '}
          <select value={userId} onChange={(e) => setUserId(e.target.value)}>
            {USERS.map((u) => (
              <option key={u.id} value={u.id}>
                {u.name} ({u.role}, {u.id})
              </option>
            ))}
          </select>
        </label>
      </header>
      <FlagProvider user={user}>
        <div className="layout">
          <Storefront />
          <AdminPanel />
        </div>
      </FlagProvider>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

`src/index.css`

```
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; background: #f4f5f7; color: #1f2328; }
button, select, input { font: inherit; }
button { padding: 6px 12px; border: 1px solid #c9ced6; border-radius: 6px; background: #fff; cursor:
pointer; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
:focus-visible { outline: 2px solid #2563eb; outline-offset: 2px; }

.app { max-width: 1280px; margin: 0 auto; padding: 16px; }
.top { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; justify-content: space-between;
}
.top h1 { font-size: 1.4rem; margin: 0; }
.layout { display: grid; grid-template-columns: minmax(280px, 380px) 1fr; gap: 16px; margin-top: 16px;
align-items: start; }
@media (max-width: 860px) { .layout { grid-template-columns: 1fr; } }

.card { background: #fff; border: 1px solid #e1e4e8; border-radius: 10px; padding: 16px; overflow-x:
auto; }
.card h2 { margin: 0 0 12px; font-size: 1.1rem; }
.muted { color: #6a737d; font-size: 0.85rem; }
.notice { background: #fff7e6; border: 1px solid #f5c26b; padding: 8px 10px; border-radius: 6px; }

.store.dark { background: #1d2127; color: #e6e6e6; border-color: #333; }
.store.dark .muted { color: #9aa4af; }
.banner { background: #e8f1ff; color: #1d4ed8; padding: 8px 10px; border-radius: 6px; margin-bottom:
10px; }
.store-nav { display: flex; gap: 12px; margin-bottom: 12px; }
.store-nav a { color: inherit; }
.product { display: flex; gap: 12px; }
.thumb { width: 80px; height: 80px; border-radius: 8px; background: linear-gradient(135deg, #a5b4fc,
#f0abfc); flex: none; }
.product h3 { margin: 0; }
.btn.primary { background: #2563eb; color: #fff; border-color: #2563eb; }
.upsell { display: block; margin-top: 8px; font-size: 0.9rem; }
.skeleton { height: 18px; border-radius: 4px; background: #eceff3; margin: 10px 0; }
.skeleton.short { width: 60%; }

.admin-head { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; margin-bottom: 12px; }
.admin-head h2 { margin: 0 auto 0 0; }
.admin-head input { padding: 6px 8px; border: 1px solid #c9ced6; border-radius: 6px; }
table { width: 100%; border-collapse: collapse; font-size: 0.9rem; }
th, td { text-align: left; padding: 8px; border-bottom: 1px solid #eef0f2; vertical-align: top; }
th { font-weight: 600; color: #57606a; }
.switch { display: inline-flex; gap: 6px; align-items: center; }
.rollout { display: inline-flex; gap: 6px; align-items: center; }
```

```
.rollout input { width: 110px; }
.pill { display: inline-block; padding: 1px 8px; border-radius: 999px; font-size: 0.75rem; font-weight: 600; }
.pill.on { background: #dcfce7; color: #166534; }
.pill.off { background: #f1f2f4; color: #57606a; }
```

`src/flags/evaluate.test.mjs` (run with `node src/flags/evaluate.test.mjs`)

```
// Run: node src/flags/evaluate.test.mjs
import assert from 'node:assert/strict';
import { bucketOf, evaluateFlag, evaluateAll } from './evaluate.js';

const user = { id: 'u-101', role: 'user' };
const flag = (over) => ({ key: 'f', enabled: true, rollout: 100, roles: [], requires: null, ...over });

// bucket is stable and in range
assert.equal(bucketOf('u-1', 'f'), bucketOf('u-1', 'f'));
for (let i = 0; i < 1000; i++) assert.ok(bucketOf(`u-${i}`, 'f') >= 0 && bucketOf(`u-${i}`, 'f') < 100);

// rollout % is roughly honoured over many users
const on = Array.from({ length: 10000 }, (_, i) => evaluateFlag(flag({ rollout: 30 }), { id: `u-${i}`, role: 'user' }).on).filter(Boolean).length;
assert.ok(on > 2700 && on < 3300, `30% rollout gave ${on}/10000`);

// order: override > kill switch > roles > rollout
assert.equal(evaluateFlag(undefined, user).on, false);
assert.equal(evaluateFlag(flag({ enabled: false }), user, true).on, true);
assert.equal(evaluateFlag(flag(), user, false).on, false);
assert.equal(evaluateFlag(flag({ enabled: false }), user).on, false);
assert.equal(evaluateFlag(flag({ roles: ['admin'] }), user).on, false);
assert.equal(evaluateFlag(flag({ roles: ['admin'] }), { id: 'x', role: 'admin' }).on, true);
assert.equal(evaluateFlag(flag({ rollout: 0 }), user).on, false);
assert.equal(evaluateFlag(flag({ rollout: 100 }), user).on, true);

// prerequisites and cycles
const parentOff = evaluateAll([flag({ key: 'a', enabled: false }), flag({ key: 'b', requires: 'a' })], user);
assert.equal(parentOff.b.on, false);
assert.match(parentOff.b.reason, /needs "a"/);
assert.equal(evaluateAll([flag({ key: 'a' }), flag({ key: 'b', requires: 'a' })], user).b.on, true);
assert.equal(evaluateAll([flag({ key: 'b', requires: 'missing' })], user).b.on, false);
const cycle = evaluateAll([flag({ key: 'a', requires: 'b' }), flag({ key: 'b', requires: 'a' })], user);
assert.equal(cycle.a.on, false);
assert.equal(cycle.b.on, false);
```

```
// a forced override skips the prerequisite check
assert.equal(evaluateAll([flag({ key: 'a', enabled: false }), flag({ key: 'b', requires: 'a' })], user,
{ b: true }).b.on, true);

console.log('evaluate.js: all checks passed');
for (const id of ['u-101', 'u-202', 'u-303', 'u-404']) console.log(id, 'new_checkout bucket',
bucketOf(id, 'new_checkout'));
```

---

12. Wrap-up

Edge cases handled

Unknown keys, loading without cache (skeleton), failure with and without cache, stale responses, optimistic rollback, prerequisites and cycles, 0% / 100% rollout, overrides winning over server rules, broken `localStorage`, hook outside provider.

Trade-offs to mention

- **Client-side evaluation** ships the whole config (including role lists and unreleased flag names) to the browser. Simple and works offline, but leaks roadmap info. Server-side evaluation returning only `{ key: boolean }` per user is safer for sensitive flags.
- **One context for everything**: any flag change re-renders all consumers. Fine for tens of flags; for hundreds, use a store with selectors (`useSyncExternalStore`) so components re-render only when *their* flag changes.
- **Hashing** with FNV-1a mod 100 gives 1% granularity and is good enough for UI rollouts; a real SDK uses a stronger hash and 10,000 buckets.
- **Default OFF** is safe for new features but wrong for kill switches of *existing* features ("disable_payments"): name flags so OFF is always the safe state, or allow per-flag defaults.
- **Rollback per field** is good, but two quick toggles of the same field can still race; a version number / `updatedAt` from the server would fix it.

What to add with more time

- Live updates via SSE / WebSocket or polling with `ETag`, without flipping UI under a user mid-task (apply on next navigation).
- Multivariate flags / A/B variants (`variant: 'A' | 'B'`) and exposure tracking events for analytics.
- More rules: user id allow-list, country, app version, date window.
- Audit log and confirmation for production changes; RBAC on the admin panel.
- Unit tests with Vitest + React Testing Library for `<Feature>`, provider error paths and the slider commit.
- Removing stale flags: a lint rule or report of flags at 100% for N days.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why evaluate flags on the client instead of asking the server for booleans?

The admin panel needs the full config anyway, client evaluation works offline and lets you switch users instantly. The cost is that rules and unreleased flag names are visible in the bundle. For sensitive flags I'd have the server evaluate and return only `{ key: on }` for the logged-in user.

Q2. What should the user see if the flag service is down?

The last known config from the cache, otherwise all flags OFF, which is the old, proven experience. The app must never block on the flag service. We also show a Retry so nobody is stuck.

Q3. Should overrides work in production?

Usually only for internal users (a query param or a cookie that the server validates), otherwise anyone can enable unfinished features from devtools. Client-side gating is never security: anything truly protected must also be checked on the server.

State design

Q4. Why not store the evaluated `results` in state?

They are a pure function of `flags`, `user` and `overrides`. Storing them means syncing three sources by hand and getting stale results when the user switches. `useMemo` derives them once per change.

Q5. Why is the current user outside the FlagProvider?

The user belongs to auth, not to the flag system. The provider receives it as a prop, so the same provider works with any auth setup and re-evaluates automatically when the prop changes.

Q6. Every flag change re-renders every `useFlag` consumer. Is that a problem?

For tens of flags, no. With hundreds of flags and big trees I'd move flags to an external store and use `useSyncExternalStore(store.subscribe, () => store.get(key))`, so a component re-renders only when its own flag changes.

Challenge: sticky rollout

Q7. Why not `Math.random() < rollout / 100`?

It changes on every render and reload, so users flip between experiences, and analytics can't attribute behavior to a variant. A hash of `flagKey:userId` is deterministic: same user, same answer, on every device.

Q8. Why include the flag key in the hash?

Without it, the same 10% of users would be early adopters for every experiment, which biases results and overloads those users with bugs. Mixing in the key gives each flag an independent split.

Q9. What about logged-out users with no id?

Generate an anonymous id once, store it in a cookie / `localStorage`, and use it for bucketing. On login, either keep the anonymous id for the session or re-bucket and accept a possible switch.

Challenge: prerequisites

Q10. What if someone configures A requires B and B requires A?

The `visiting` set detects the cycle and returns OFF with reason "circular prerequisite" instead of overflowing the stack. Better still, the admin API should reject cycles on save.

Q11. Why does an override skip the prerequisite check?

An override is an explicit "I want this state" from a developer testing one piece. If they need the parent too, they can override it as well. It keeps overrides predictable.

Challenge: loading and flicker

Q12. How do you avoid the flash of the old UI on first load?

Start from cached flags so returning users render correctly on the first frame. For a first-ever visit, render a skeleton for gated areas instead of guessing. With SSR, the server evaluates flags and inlines them in the HTML, so there is no flash at all.

Q13. Two fetches overlap (StrictMode, double click on Refresh). What happens?

Each fetch gets an incrementing request id; when a response arrives, it's dropped if a newer request has started. Without this, a slow old response could overwrite fresh data. An `AbortController` would also cancel the network request.

Challenge: optimistic updates

Q14. Why roll back only the changed fields instead of the whole previous flag?

If the admin changed `rollout` and then `enabled` on the same flag and only the first request fails, restoring the whole old flag would also undo the second, successful change. Restoring only the patched fields keeps the other edit.

Q15. Why not debounce the slider instead of committing on release?

Debounce works too, but it can fire mid-drag if the admin pauses. Commit on pointer up / key up / blur matches intent: one request per decision. Keyboard users still commit on each key release.

Implementation steps

Q16. Why a `<Feature>` component and a `useFlag` hook? Isn't one enough?

`<Feature>` is readable for swapping JSX blocks; `useFlag` is needed for logic that isn't JSX (class names, which API to call, analytics). The component is a one-liner on top of the hook, so there is no duplication.

Q17. How would you remove a flag once a feature is fully launched?

Set it to 100% for a while, then delete the `useFlag` / `<Feature>` call sites and the fallback code, then delete the flag on the server. Searching for the key string finds every call site, which is one reason keys are plain string constants.

Performance and "what if"

Q18. What if there are 5,000 flags?

Evaluation is O(flags) and still fast, but don't ship all of them: fetch only flags for this app / page, virtualize the admin table, and use a selector-based store so changing one flag doesn't re-render everything.

Q19. What if a flag must change for all open tabs within seconds (incident kill switch)?

Push changes via SSE / WebSocket (or short polling with ETag). Apply kill switches immediately; for cosmetic flags, apply on the next navigation so the UI doesn't change under the user. A `storage` event can sync tabs of the same browser.

Q20. What if the API is slow (5 s)?

Cached flags make it invisible for returning users. For new users, render the default experience after a short timeout (e.g. 1 s) instead of a long skeleton, and apply real flags on the next navigation to avoid a mid-session swap.

Trade-offs

Q21. Why Context and not Redux / Zustand?

The state is small, read-mostly and app-wide, which is exactly what Context is for, and it adds no dependency. If re-render cost or devtools needs grow, the provider's API (`useFlag` , `updateFlag`) stays the same and only its internals change.

Q22. Why cache flags in `localStorage` and not just refetch?

It removes the flicker for returning users and lets the app work offline or when the flag service is down. The risk is a stale config for one load, which the background refetch fixes immediately.